



PY32F003 系列

32 位 ARM® Cortex®-M0+ 微控制器

HAL 库样例手册

PY32F003 系列

32 位 ARM® Cortex®-M0+ 微控制器

HAL 库样例手册

1 ADC

1.1 ADC_DMA

此样例演示了通过 ADC 触发 DMA 搬运 ADC_DR 中的数据,配置 CH0(PA00)为 ADC 的模拟输入通道, ADC 触发 DMA 搬运采样数据, DMA 搬运完成,在循环中打印搬运的数据

1.2 ADC_MultiChannelSingleConversion

此样例演示了 ADC 单次轮询(Polling)的方式经行采样 AN0,AN1,AN4 通道

1.3 ADC_MultiChannelSingleConversion_AWD

此样例演示了 ADC 的模拟看门狗功能.

1.4 ADC_TempSensor

此样例演示了 ADC 模块的温度传感器功能,通过仿真器,在中断根据观测采样值 (aADCxConvertedData),计算出当前的温度。

1.5 ADC_Timer

此样例演示了通过 TIM1 触发 ADC 模块的通道采样功能,配置 CH0(PA00)为 ADC 的模拟输入通道, TIM1 配置为主模式, TIM1 每产生一次更新事件,触发一次 ADC 采样,采样数据在中断中打印

1.6 ADC_Timer_DMA

此样例演示了通过 TIM1 触发 ADC,再通过 ADC 触发 DMA 搬运 ADC_DR 种的数据,配置 CH0(PA00)为 ADC 的模拟输入通道, TIM1 配置为主模式, TIM1 每产生一次更新事件,触发一次 ADC 采样,然后 ADC 触发 DMA 搬运采样数据,采样数据在中断中打印

1.7 ADC_VREFINT

此样例演示了 ADC 模块的 VCC 采样功能,通过采样 VREFINT,反推 VCC 电压

2 COMP

2.1 COMP1_INT

此样例演示了 COMP 比较器功能，PA01 作为比较器正端输入，VREFINT 作为比较器负端输入，PA06 作为比较器的输出端口，通过调整 PA01 上的输入电压，观测 PA06 引脚上的电平变化和 PB5 上的电压翻转

2.2 COMP1_POLLING

此样例演示了 COMP 比较器功能，PA01 作为比较器正端输入，VREFINT 作为比较器负端输入，PA06 作为比较器的输出端口，通过调整 PA01 上的输入电压，观测 PA06 引脚上的电平变化。

2.3 COMP1_Wakeup

此样例演示了 COMP 比较器唤醒功能，PA01 作为比较器正端输入，VREFINT 作为比较器负端输入，PA06 作为比较器的输出端口，进入 stop 模式后，通过调整 PA01 上的输入电压，产生中断唤醒 stop 模式。

2.4 COMP1_WINDOW

此样例演示了 WINDOW 比较器功能，PA01 作为比较器正端输入，内部通过 COMP1 和 COMP2 的正极相连，PA1 输入大于 VREF 并且大于 $1/4VREF$ 的电压，LED 以 200ms 的间隔进行翻转；PA1 输入小于 VREF 并且小于 $1/4VREF$ 的电压，LED 熄灭；PA1 输入的电压小于 VREF 大于 $1/4VREF$ ，LED 常亮

2.5 COMP2_HYST

此样例演示了 COMP 比较器迟滞功能，PA03 作为比较器正端输入，VREFINT 作为比较器负端输入，PA07 作为比较器的输出端口，通过调整 PA03 上的输入电压，观测 PA07 引脚上的电平变化。

3 CRC

3.1 CRC_Computing_Results

此样例演示了 CRC 校验功能，通过对一个数组里的数据进行校验，得到的校验值与理论校验值进行比较，相等则 LED 灯亮，否则 LED 灯熄灭。

4 DMA

4.1 DMA_SramToSram

此样例演示了 DMA 从 SRAM 到 SRAM 传输数据的功能（SRAM 和外设之间传输的样例请参考相关外设样例工程），数据传输完成 LED 灯闪烁，传输失败则 LED 灯熄灭

5 EXTI

5.1 EXTI_Event

此样例演示了通过 PA12 引脚唤醒 MCU 的功能，下载程序后，MCU 会进入 STOP 模式，按下用户键 MCU 会退出 STOP 模式，然后 LED 会以 500ms 的间隔进行翻转

5.2 EXTI_IT

此样例演示了 GPIO 外部中断功能，PA12 引脚上的每一个下降沿都会产生中断，中断函数中 LED 灯会翻转一次

6 FLASH

6.1 FLASH_OptionByteWrite_RST

此样例演示了通过软件方式将 RESET 引脚改为普通 GPIO

6.2 FLASH_PageEraseAndWrite

此样例演示了 flash page 擦除和 page 写功能，通过 keil debug 仿真界面，可观察 flash 存储器中是否擦除成功和 page 写成功。

6.3 FLASH_SectorEraseAndWrite

此样例演示了 flash sector 擦除和 page 写功能，通过 keil debug 仿真界面，可观察 flash 存储器中是否擦除成功和 page 写成功。

7 GPIO

7.1 GPIO_FastIO

本样例主要展示 GPIO 的 FAST IO 输出功能。FAST IO 速度可以达到单周期翻转速度；

7.2 GPIO_Toggle

此样例演示了 GPIO 输出模式，配置 LED 引脚(PA11)为数字输出模式，并且每隔 250ms 翻转一次 LED 引脚电平，运行程序，可以看到 LED 灯以 2Hz 的频率闪烁

8 I2C

8.1 I2C_TwoBoard_CommunicationMaster_DMA

此样例演示了从机 I2C 通过 DMA 方式进行通讯，主机使用 PY32F003，按下 user 按键，主机先向从机发送 15 byte 数据，然后再接收从机发送的数据，数据接收后，会保存在 aRxBuffer 数组中，接收数据为 0x1~0xf，以上发送和接收的过程会一直循环。

8.2 I2C_TwoBoard_CommunicationMaster_DMA_MEM

此样例演示了主机 I2C 通过 DMA 方式进行通讯，从机使用 EEPROM 外设芯片 P24C32，按下 user 按键，主机先向从机写 15 bytes 数据为 0x1~0xf，然后再从 EEPROM 中读取数据，数据读取后，会保存在 aRxBuffer 数组中，接收数据为 0x1~0xf。

8.3 I2C_TwoBoard_CommunicationMaster_IT

此样例演示了主机 I2C 通过中断方式进行通讯，从机使用 PY32F003，按下 user 按键，主机先向从机发送 15 byte 数据，然后再接收从机发送的数据，数据接收后，会保存在 aRxBuffer 数组中，接收数据为 0x1~0xf。

8.4 I2C_TwoBoard_CommunicationMaster_Polling

此样例演示了主机 I2C 通过 POLLING 方式进行通讯，从机使用 PY32F003，按下 user 按键，主机先向从机发送 15 byte 数据，然后再接收从机发送的数据，数据接收后，会保存在 aRxBuffer 数组中，接收数据为 0x1~0xf。

8.5 I2C_TwoBoard_CommunicationSlave_DMA

此样例演示了从机 I2C 通过 DMA 方式进行通讯，主机使用 PY32F003，按下 user 按键，主机先向从机发送 15 byte 数据，然后再接收从机发送的数据，数据接收后，会保存在 aRxBuffer 数组中，接收数据为 0x1~0xf，以上发送和接收的过程会一直循环。

8.6 I2C_TwoBoard_CommunicationSlave_IT

此样例演示了主机 I2C 通过中断方式进行通讯，从机使用 PY32F003，按下 user 按键，主机先向从机发送 15 byte 数据，然后再接收从机发送的数据，数据接收后，会保存在 aRxBuffer 数组中，接收数据为 0x1~0xf。

9 IWDG

9.1 IWDG_Reset

此样例演示了 IWDG 看门狗功能，配置看门狗重载计数值，计数 1s 后复位，然后通过调整每次喂狗的时间（main 函数 while 循环中代码），可以观察到，如果每次喂狗时间小于 1s 钟，程序能一直正常运行（LED 灯闪烁），如果喂狗时间超过 1s 钟，程序会一直复位（LED 灯熄灭）。

10 LPTIM

10.1 LPTIM_Wakeup

此样例演示了 LPTIM 中断唤醒 stop 模式，每次唤醒后再次进入 stop 模式，每 200ms 唤醒一次。

11 PWR

11.1 PVD

此样例演示了 PVD 电压检测功能，样例中配置 PB07 引脚的电压与 VREF(1.2v)进行比较，当 PB07 引脚的电压高于 VREF 时,LED 灯灭，当低于 VREF 时，LED 灯亮，

11.2 PWR_SLEEP_WFE

此样例演示了 sleep 模式下，通过 GPIO 事件唤醒功能

11.3 PWR_SLEEP_WFI

此样例演示了 sleep 模式下，GPIO 外部中断唤醒功能

11.4 PWR_STOP_WEF

此样例演示了 stop 模式下，通过 GPIO 事件唤醒功能

11.5 PWR_STOP_WFI

此样例演示了 stop 模式下，GPIO 外部中断唤醒功能

12 RCC

12.1 HSI_CALIBRATION

此样例校准内部 HSI

12.2 RCC_HSE_Div

此样例配置系统时钟为 HSE，并通过 MCO (PA01) 引脚输出

12.3 RCC_HSI_Output

此样例配置系统时钟为 HSI，并通过 MCO (PA01) 引脚输出

12.4 RCC_LSI_Output

此样例配置系统时钟为 LSI，并通过 MCO (PA01) 引脚输出，注意系统时钟切换为 LSI 之前，要求把 systick 中断关闭掉，因为 systick 中断默认是 1ms 一次中断，由于 LSI 时钟频率过低，systick 中断会导致程序无法正常运行。

12.5 RCC_Sysclock_Switch

此样例演示系统时钟切换功能，样例中配置系统时钟从 LSI 切换到 HSE，并通过 MCO (PA01) 引脚输出系统时钟。

13 RTC

13.1 RTC_AlarmSecond_IT

此样例演示 RTC 的秒中断和闹钟中断功能，每次秒中断，在中断函数中会打印字符“RTC_IT_SEC”，并且输出实时时间。

13.2 RTC_WeakUpSecond

此样例演示 RTC 的秒中断和闹钟中断功能，每次秒中断，在中断函数中会打印字符“RTC_IT_SEC”，并且输出实时时间，并且将 MCU 从 STOP 模式中唤醒。

14 SPI

14.1 SPI_TwoBoards_FullDuplexMaster_DMA

此样例是对 串口外设接口(SPI)与外部设备以全双工串行方式进行通信 的演示,此接口设置为主模式,为外部从设备提供通信时钟 SCK。主机通过 MOSI 引脚发送数据,从 MISO 引脚接收从机的数据,数据以主机提供的 SCK 沿同步被移位,完成全双工通信。

14.2 SPI_TwoBoards_FullDuplexSlave_DMA

此样例是对 串口外设接口(SPI)与外部设备以全双工串行方式进行通信 的演示,此接口设置为从模式,通信时钟 SCK 由外部主设备提供。从机通过 MOSI 引脚接收主机数据,从 MISO 引脚发送数据,数据以主机提供的 SCK 沿同步被移位,完成全双工通信。

15 TIMER1

15.1 tim1_ARR

此样例实现了定时器的基本计数功能，以及演示了 ARR 自动重载功能，样例在定时器重载中断中翻转 LED 灯 修改 main.c 中的第 42 行配置 TimHandle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;使能自动重载功能，新的 ARR 值在第四次进中断时生效，配置 TimHandle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;禁止自动重载功能，新的 ARR 值在第三次进中断时生效,生效后，LED 灯每隔 400ms 翻转一次

15.2 tim1_ComplementarySignals

此样例实现了定时器的互补输出功能，一组互补共两路 pwm 输出，此样例没有实现死区功能 CH1 -> PA3CH1N -> PA0

15.3 tim1_ComplementarySignals_break

此样例实现了定时器的刹车功能，CH1 和 CH1N 互补 pwm 输出，接收到外部 IO 口的刹车信号（低电平）后，PWM 信号关闭，由于 BDTR.AOE 置位，所以刹车信号取消（高电平）后，继续 pwm 输出，此样例实现了死区功能。CH1 -> PA3CH1N -> PA0 刹车输入 -> PA6 通过调整 OCxE,CCxP,OISx,CCxNE,CCxNP,OISxN 的配置，可实现刹车功能的各种应用

15.4 tim1_ComplementarySignals_break_it

此样例实现了定时器的刹车功能，CH1 和 CH1N 互补 pwm 输出，接收到外部 IO 口的刹车信号（低电平）后，PWM 信号关闭，由于 BDTR.AOE 置位，所以刹车信号取消（高电平）后，继续 pwm 输出，此样例实现了死区功能。本样例开启了刹车中断，并在刹车中断里翻转 LED 灯 CH1 -> PA3CH1N -> PA0 刹车输入 -> PA6 通过调整 OCxE,CCxP,OISx,CCxNE,CCxNP,OISxN 的配置，可实现刹车功能的各种应用

15.5 tim1_ComplementarySignals_DeadTime

此样例实现了定时器的刹车功能，CH1 和 CH1N 互补 pwm 输出，接收到外部 IO 口的刹车信号（低电平）后，PWM 信号关闭，由于 BDTR.AOE 置位，所以刹车信号取消（高电平）后，继续 pwm 输出，此样例实现了死区功能。CH1 -> PA3CH1N -> PA0 刹车输入 -> PA6 通过调整 OCxE,CCxP,OISx,CCxNE,CCxNP,OISxN 的配置，可实现刹车功能的各种应用

15.6 tim1_dma

此样例演示了在 TIM1 中使用 DMA 传输数据的功能，通过 DMA 从 SRAM 中搬运数据到 ARR 寄存器，实现 TIM1 周期变化，在 TIM1 第一次溢出后，PA0 会翻转，此时翻转间隔为 400ms，DMA 开始搬运数据到 TIM1_ARR，第二次 PA0 翻转间隔为 400ms，第三次翻转间隔为 100ms，第四次翻转间隔为 200ms，第四次翻转间隔为 300ms，此时 DMA 搬运结束，后续翻转间隔均为 300ms

15.7 tim1_dma_burst

此样例演示了在 TIM1 中使用 DMA burst 传输数据的功能，burst 传输一次更新三个寄存器，PSC,ARR,RCR，在更新事件中断中，PA0 会进行翻转，通过逻辑分析仪监测，可看到 PA0 的翻转间隔会从第一次 400ms，第二次 400ms，第三次及后续变为 10ms，此时 PCS,ARR,RCR 均更新完毕。

15.8 tim1_dma_burst_twice

此样例演示了在 TIM1 中使用 DMA 连续两次 burst 传输数据的功能，burst 每传输一次更新三个寄存器，PSC,ARR,RCR，在更新事件中断中，PA0 会进行翻转，通过逻辑分析仪监测，可看到 PA0 的翻转间隔会从第一次的 400ms，第二次 400ms，第三次 10ms，第四次及后续变为 100us，此时两次 burst 传输完成，并且 PCS,ARR,RCR 均更新完毕。

15.9 tim1_externalclockmode1

此样例演示了 TIM1 的外部时钟模式 1 功能，选择 ETR(PA12)引脚作为外部时钟输入源，并使能更新中断，在中断中翻转 LED 灯

15.10 tim1_externalclockmode2

此样例演示了 TIM1 的外部时钟模式 2 功能，选择 ETR(PA12)引脚作为外部时钟输入源，并使能更新中断，在中断中翻转 LED 灯

15.11 tim1_InputCapture_TI1FP1_CH1

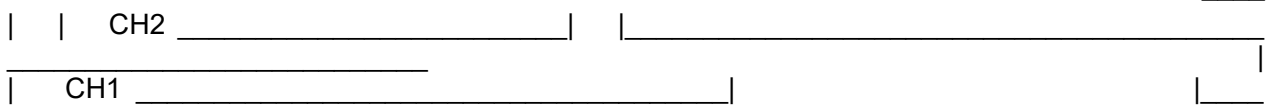
此样例演示了在 TIM1(PA3)输入捕获功能，PA8 输入时钟信号，TIM1 捕获成功后，会进入捕获中断，每进一次中断，翻转一次 LED

15.12 tim1_oc_4ch_toggle

此样例演示了 TIM1 比较模式下的 OC 翻转输出功能,使能 CH1(PA3),CH2(PA13),CH3(PA0),CH4(PA1) 四个通道的输出功能,并且当计数器 TIMx_CNT 与 TIMx_CCRx 匹配时输出信号翻转,周期为 200KHz

15.13 tim1_OnePulse

此样例演示了 TIM1 的单脉冲模式, CH2(PA13)引脚上的上升沿,触发计数器开始计数,当计数值与 CCR1 匹配时, CH1(PA3)输出高电平,直到计数器溢出, CH1 再次输出低电平,计数器溢出后,定时器停止工作,输出效果见下图:



<---Delay---><-----Pulse----->本例程脉冲宽度计算 $(TIM1_ARR-TI1_CCR1)/CLK = (65535-16383) / 16000000 = 3.072ms$

15.14 tim1_synchronization_enable

定时器 1 的使能由定时器 3 控制,当定时器 3 计数时,LED 会常亮,当定时器 3 发生更新事件时,更新事件会触发定时器 1,定时器 1 开始计数后,LED 会以 5Hz 的频率进行翻转

15.15 tim1_tim3_cascade

此样例实现了 TIM1 和 TIM3 级联成 32 位计数器, TIM3 做主机, TIM3 的计数溢出信号作为 TIM1 的输入时钟,通过配置 TIM1 和 TIM3 的重载寄存器值,(在 TIM1 中断回调函数中)实现 LED 灯以 0.5Hz 频率闪烁此例程计算方式为 $TIM3_ARR * TIM3_PSC * TIM1_ARR * TIM1_PSC / \text{时钟} = 800 * 100 * 100 * 1 / 8000000 = 1Hz$

15.16 tim1_update

此样例演示了在 TIM1 中基本计数功能,并使能了更新中断,每次重装 ARR 值时会产生一次更新中断,并在中断中翻转 LED 灯,LED 灯会以 5Hz 的频率进行翻转。

16 TIMER16

16.1 TIMER16_Counter

此样例演示了在 TIM16 中基本计数功能,并使能了更新中断,每次重装 ARR 值时会产生一次更新中断,并在中断中翻转 LED 灯,LED 灯会以 5Hz 的频率进行翻转。

17 TIMER17

17.1 TIMER17_Counter

此样例演示了在 TIM17 中基本计数功能,并使能了更新中断,每次重装 ARR 值时会产生一次更新中断,并在中断中翻转 LED 灯,LED 灯会以 5Hz 的频率进行翻转。

18 TIMER3

18.1 tim3_dma_capture

此样例演示了在 TIM3 输入捕获功能，捕获数据通过 DMA 传输到变量 CC1_Capture 中，并通过串口打印输出。UART_TX->PA2UART_RX->PA3

18.2 tim3_slavegatedmode_CH1

此样例演示了 TIM3 从模式下的门控触发功能，配置 CH1(PA06)作为门控触发输入信号，并使能触发中断，每次进触发中断，翻转一次 LED 灯，在 IDE 仿真界面里，可以看到，当 PA06 输入低电平时，定时器 CNT 寄存器停止计数，当 PA06 输入高电平时，定时器 CNT 寄存器持续在计数。并且 PA06 引脚上的每产生一次触发中断，LED 都会翻转一次

18.3 tim3_synchronization_compare

此样例演示了 TIM1 和 TIM3 同步触发（门控触发模式）功能，TIM1 作为主机，配置为比较输出功能，OC1 输出频率 1kHz，占空比 50%（即 1ms 高电平 1ms 低电平循环），TIM3 作为从机，配置为外部门控触发模式，TIM1 的 OC1 信号连接到 TIM3 作为 TIM3 的门控触发输入，配置 TIM3 的计数溢出周期为 125us，并允许溢出中断，且在溢出中断中对 LED 灯翻转。通过运行程序，通过逻辑分析仪可以看到，当 TIM1_CH1(PA3)输出高电平时，LED(PB5)电平翻转；当 TIM1_CH1(PA3)输出低电平时，LED(PB5)电平不翻转

18.4 tim3_synchronization_reset

此样例演示了 TIM1 和 TIM3 同步触发（复位触发模式）功能，TIM1 作为主机，配置复位 TRGO 输出 (TIM1_CR2.MMS=000)，TIM3 作为从机并且使能事件更新中断，配置收到主机的 TRGO 信号后复位 (TIM3_SMCR.SMS=100)，仿真模式下查看 TIM3 会清空 CNT 的值，然后从 0 开始重新计数，不会产生更新中断，LED 不会翻转

18.5 tim3_update

此样例演示了在 TIM3 中基本计数功能，并使能了更新中断，每次重装 ARR 值时会产生一次更新中断，并在中断中翻转 LED 灯，LED 灯会以 5Hz 的频率进行翻转。

19 USART

19.1 USART_HyperTerminal_AutoBaud_IT

此样例演示了 USART 的自动波特率检测功能，调试助手发送一个字符 0x7F，MCU 反馈字符串：Auto BaudRate Test。

19.2 USART_HyperTerminal_DMA

此样例演示了 USART 的 DMA 方式发送和接收数据，USART 配置为 115200，数据位 8，停止位 1，校验位 None,下载并运行程序后，通过 USART 会接收到 0x1-0xC,然后通过上位机下发 12 个数据，例如 0x1~0xC,则，MCU 会把接收到的数据再次发送。

19.3 USART_HyperTerminal_IT

此样例演示了 USART 的中断方式发送和接收数据，USART 配置为 115200，数据位 8，停止位 1，校验位 None,下载并运行程序后,上位机通过 USART 会接收到 0x1-0xC,然后通过上位机下发 12 个数据，例如 0x1~0xC,则，MCU 会把接收到的数据再次发送到上位机，后续 USART 循环上面的收发。

19.4 USART_HyperTerminal_Polling

此样例演示了 USART 的 POLLING 方式发送和接收数据，USART 配置为 115200，数据位 8，停止位 1，校验位 None,下载并运行程序后，通过 USART 会接收到 0x1-0xC,然后通过上位机下发 12 个数据，例如 0x1~0xC,则，MCU 会把接收到的数据再次发送。

20 WWDG

20.1 WWDG_IT

此样例演示了 WWDG 的 提前唤醒中断 功能，看么狗计数器向下计数到 0x40 时产生中断，中断中喂狗，可以确保看门狗不会复位。

20.2 WWDG_Window

此样例演示了 WWDG 的 窗口看门狗功能，配置 WWDG 的窗口上线（下限固定是 0x3F），程序中通过 delay 延时函数，确保程序是在 WWDG 计数窗口内进行喂狗动作，通过 LED 灯闪烁，可以判断窗口内喂狗并未产生复位。